

Tema 0-a Expresiones Regulares

Investigación en Ciencia de Datos
(Máster en DS & BI)

José Fernández Menéndez

Departamento de Organización de Empresas - UCM

Índice

- 1 Introducción
- 2 Uso básico de regex
- 3 Metacaracteres
- 4 Flags en regex
- 5 Clases de caracteres
- 6 Paréntesis en regex
- 7 Cuantificadores
- 8 Funciones con *regex*
- 9 Operadores lookahead
- 10 Ejemplo

Introducción

- Una **expresión regular** (*regular expression* o *regexp*, o *regex*) es un *string* (luego irá entre comillas) que permite indicar **patrones de caracteres**.
- En R, y en otros muchos lenguajes, existen funciones que toman como argumentos un texto y una expresión regular y permiten hacer cosas como:
 - ▶ detectar si el patrón indicado por la *regexp* está presente en el texto (puede estar presente una o varias veces)
 - ▶ indicar la posición de esa ocurrencia u ocurrencias (puede que se indique sólo la primera o todas)
 - ▶ extraer las ocurrencias
 - ▶ reemplazar las ocurrencias por otros caracteres
- Las expresiones regulares comienzan a desarrollarse en torno a 1950 y se consolidan a principios de la década de 1970 con la aparición del comando *grep*, habitual en Unix y Linux, para la búsqueda de patrones de texto dentro de archivos. En Windows *Power Shell* un comando más o menos equivalente a *grep* es *Select-String*.

Introducción

- Las *regex* no llegan a ser un lenguaje de programación, pero tienen una notable potencia y complejidad
- P. ej en <https://pdw.ex-parrot.com/Mail-RFC822-Address.html> se puede ver una *regex* que, se supone, sirve para chequear si una dirección de correo electrónico es válida.
- Existen diversas variantes de las expresiones regulares, aunque todas son similares.
- Las tres variantes clásicas son «basic» (BRE), «extended» (ERE) y «perl» (PCRE), que es la versión usada en perl, un lenguaje de programación especialmente potente para el manejo de caracteres.
- En Wikipedia, en el [artículo sobre expresiones regulares](#) se puede encontrar una buena cantidad de información y de bibliografía.
- Una referencia clásica, considerada "la biblia" de las *regex* es Friedl (2006), pero hay un gran número de libros y webs con información interesante.

Introducción

- Existen **diversas librerías**, denominadas habitualmente *engines*, motores, que implementan expresiones regulares; entre estos motores, están por ejemplo, **PCRE**, **TRE**, o **ICU**, del consorcio Unicode.
- En R base se usan dos variantes de *regex*:
 - ▶ ERE, por defecto, que se basa en el **estándar POSIX 1003.2** (POSIX es un conjunto de estándares desarrollados por el **IEEE** para intentar conseguir la compatibilidad entre sistemas operativos), y está implementada usando el motor TRE
 - ▶ perl, con la opción perl=TRUE, que hace que se use el motor PCRE
- Aquí vamos a ver el funcionamiento de las *regex* tal y como están implementadas en el paquete *stringi* (y por lo tanto en *stringx*), que se basa en el motor ICU, cuya sintaxis está descrita en **<https://www.unicode.org/reports/tr18/>**

Uso básico de regex

- En su uso básico la función *grep* de Unix toma una patrón de texto indicado por una expresión regular y un **fichero** de texto, y devuelve las líneas del fichero en las que se encuentra ese patrón.
- En R base se usa *grep()* o *grepl()* para detectar si un *string* contiene el patrón de texto indicado por una expresión regular.
- En *stringx* se usa *grepl2()*, cuyos dos primeros argumentos son *x*, un vector de *strings*, y *pattern*, un vector de caracteres que contiene las *regexs* que serán buscadas dentro de los *strings* para ver si están presentes o no.
- *grepl2()* devuelve un vector lógico de la misma longitud que *x* indicando para cada elemento si hay "match" o no con el correspondiente patrón:

```
> grepl2(c("casa","coche"),"as")# se recicla el pattern
[1] TRUE FALSE
> grepl2(c("casa","coche"),c("es","oc"))
[1] FALSE TRUE
```

Uso básico de regex

- La función `stri_extract_all_regex()` de *stringi* devuelve los *matches* (hay otras funciones dentro de *stringx* que permiten hacer esto):

```
> stri_extract_all_regex(c("casa", "coche"), c("es", "oc"))
[[1]]
[1] NA
[[2]]
[1] "oc"
```

- En los ejemplos anteriores "as", "es" y "oc" son cada uno una expresión regular que hace que se busque el patrón que coincide exactamente con ella misma (es decir, se busca "as", p. ej., en el texto), pero lo interesante es que podemos indicar patrones más generales.
- Para ello se usan una serie de **meta-caracteres**, de **operadores**, de **flags** que modifican el comportamiento de la *regex*, y de **clases** (grupos) de caracteres.

Uso básico de regex

- **Meta-caracteres importantes:**

- . representa cualquier carácter salvo *newline*
- ^ representa el comienzo de línea (o de *string*)
- \$ representa el fin de línea (o de *string*); ^ y \$ se suelen denominar *anchors* (anclas)

```
> grepl2(c("casa", "coche"), c("^cas", "oc$"))
[1] TRUE FALSE
> grepl2(c("casa", "coche"), c("^cas", "he$"))
[1] TRUE TRUE
```

- Cuando queremos que un meta-carácter de una *regex* se interprete de forma literal (p. ej, que "^" sea interprete como el carácter "^", y no como un meta-carácter) hay que anteponerle el carácter de escape "\" (esto en general, pero en R hay que anteponer dos *backslashes*, "\\"):

```
> grepl2(c("5.00", "5x00"), ".00")
[1] TRUE TRUE
> grepl2(c("5.00", "5x00"), "\\ .00")
[1] TRUE FALSE
```


Metacaracteres

- Más meta-caracteres:
- `\s` indica un espacio en blanco; `\S` indica cualquier carácter que no sea espacio en blanco:

```
> grep12("casa grande",c("\\sgran","sa\\S"))
[1] TRUE FALSE
```
- `\d` indica un número (de 0 a 9); `\D` indica un no número
- `\w` indica un word character (básicamente letras y números); `\W` indica un non-word character
- `\b` indica un **borde de palabra** (*word boundary*); `\B` indica no borde de palabra (un borde de palabra significa un carácter que forma parte de una palabra (un `\w`), como una letra o un dígito, al lado de un carácter que no forma parte de una palabra (un `\W`), como un espacio en blanco):

```
> grep12("casa grande2",c("sa\\b","\\Bgran","2\\b"))
[1] TRUE FALSE TRUE
```

Metacaracteres

- En una expresión se pueden indicar grupos de caracteres poniéndolos entre corchetes: "[afH]".

- En este caso el "match" es con cualquiera de los caracteres:

```
> grepl2("semáforo",c("[afH] ","[xtg] "))
[1] TRUE FALSE
```

- \A indica el comienzo de un *string*; \Z indica el fin de de un *string*, que puede ir seguido por un *newline*; \z indica el fin (el "very end") del *string*:

```
> grepl2("Manolo el del bombo","\\AMan")
[1] TRUE
> grepl2(c("mi\ncasa","casa","mi\ncasa\n"),c("mi\\Z","sa\\Z","sa\\Z"))
[1] FALSE TRUE TRUE
> grepl2(c("mi\ncasa","casa","mi\ncasa\n"),c("mi\\z","sa\\z","sa\\z"))
[1] FALSE TRUE FALSE
```

- \A, \Z y \z son muy similares a las anclas $\wedge$ y \$, pero con la diferencia de que no se ven afectados por el *flag multiline*, que comentaremos a continuación.

Flags en regex

- Una *regex* puede llevar una serie de *flags* que modifican su comportamiento, p.ej el *flag* *(?i)* indica que a partir de ahí los *matches* son *case insensitive* (no se diferencia mayúsculas y minúsculas). Esto se anula con *(?-i)*:

```
> grep12("GRANDE",c("de","(?i)de","(?i)d(?-i)e"))
[1] FALSE TRUE FALSE
```

- Otro flag es *(?m)*, el *flag multiline*, que modifica el comportamiento de $\wedge$ y $\$$, pero no el de $\backslash A$, $\backslash Z$ y $\backslash z$, e indica si un carácter *newline* dentro de un *string* da lugar a un final y un comienzo de línea identificables con $\wedge$ y $\$$.
- P. ej en el *string* "coche\nbonito" sólo hay un comienzo de línea justo antes de "co", y un fin de línea justo después de "to", pero con el *flag multiline* activado habrá un fin de línea tras "che" y un comienzo antes de "bo".

Flags en regex

```
> grepl2("coche\nbonito",c("^bo","to","(?m)^bo"))
[1] FALSE  TRUE  TRUE
```

- Esto es en cierta medida similar a lo que ocurre con *print* y *cat* al imprimir un *string* (*print* corresponde a no *multiline* y no reconoce los caracteres `\n` mientras que *cat* *cat* sí) reconoce los `\n` y se corresponde con *multiline*):

```
> print("coche\nbonito")
[1] "coche\nbonito"
> cat("coche\nbonito\n")
coche
bonito
```

Clases de caracteres

- En una *regex* podemos usar clases (grupos, *sets*) de caracteres, de forma que el *match* se puede conseguir con **cualquier** carácter del grupo.
- La forma mas inmediata de hacerlo es, como ya se ha indicado, poniendo los caracteres del grupo entre corchetes cuadrados: "[abCD]" haría *match* con cualquiera de los 4 caracteres en su interior, «a», «b», «C» o «D».
- Dentro de los corchetes se pueden usar rangos de caracteres, p. ej "[a-h]", o "[0-3]", que indican todos los caracteres desde «a» hasta «h» (de acuerdo con la ordenación de Unicode) y los dígitos de 0 a 3, respectivamente. Puede haber múltiples combinaciones de este tipo dentro de los corchetes, p. ej. "[a-h1-5AEIOU]", que indica todas las minúsculas de «a» hasta «h», los dígitos de 1 a 5 y las vocales en mayúsculas.
- Para indicar un *match* con cualquier carácter que NO esté en la clase se pone un símbolo "^" justo tras el primer corchete. P. ej. "[^aeiou]" indica cualquier carácter que no sea una vocal en minúscula.

Clases de caracteres

- Si queremos incluir dentro de la clase los caracteres "[", "]", "-", "\", o "^", es necesario «escaparlos» anteponiéndoles "\" (en R "\\"):


```
> stri_extract_all_regex("el símbolo ^ es como un
    tejadillo", "[\\^0-9]")
```

```
[[1]]
[1] "^"
```

- De todas formas, aunque los corchetes cuadrados para definir clases se usan enormemente, pueden no ser una buena idea, como nos indica el siguiente ejemplo (una «e» con una tilde no está dentro de la clase "[a-z]", lo que no parece muy razonable):


```
> grep12("é", "[a-z]")
```

```
[1] FALSE
```

- Es preferible utilizar clases definidas por Unicode, de las que hay un gran número y para lo que el motor ICU es especialmente adecuado.

Clases de caracteres

- Otro tipo de clases muy usadas, y que tampoco son totalmente recomendables, ya que su contenido exacto depende de cada *locale*, son las clases POSIX, que tienen un formato como el de los siguientes ejemplos:

`[[:alpha:]]` indica cualquier letra (carácter alfabético) y equivale más o menos a `[a-zA-Z]`

`[[:digit:]]` indica cualquier dígito y equivale a `[0-9]`

```
> grep12(c("é", "23"), "[[:alpha:]]")
[1] TRUE FALSE
```

- Para usarlas en R hay que ponerlas dentro de los corchetes cuadrados, para indicar que se trata de una clase de caracteres.
- Se pueden ver las clases POSIX en la ayuda de R para «regex», o p. ej, en [este enlace](#).

Clases de caracteres

- Las clases definidas por Unicode sí son totalmente portables e independientes del *locale*.
- La referencia fundamental donde aparecen descritas es el **Anexo 44 de Unicode**, que es un tanto farragoso para leer.
- En general los *codepoints* de Unicode tienen determinadas propiedades y pertenecen de acuerdo con ello a determinadas categorías, que definen clases de caracteres.
- P. ej. la categoría «Letter», que se abrevia a «L», y a la que podemos hacer referencia en una regex con `\p{L}` o `\p{Letter}` o `[\p{L}]` o `[\p{Letter}]`, que indica «todos los caracteres que tienen la propiedad de ser letras».
- La clase complementaria, es decir, la de todos los caracteres que no son letras, la indicamos con un `"^"` al comienzo de una clase entre corchetes, o usando una `"P"` mayúscula: `\P{Letter}` o `[^\p{L}]`.

Clases de caracteres

- Se pueden ver diferentes categorías de Unicode en [este enlace](#) o en [este otro](#).
- Algunas de las más relevantes son:
 - ▶ `\p{L}` o `\p{Letter}`, una letra en cualquier idioma
 - ▶ `\p{Ll}` o `\p{Lowercase_Letter}`, letras minúsculas
 - ▶ `\p{Lu}` o `\p{Uppercase_Letter}`, letras mayúsculas
 - ▶ `\p{Z}` o `\p{Separator}`, espacios en blanco o separadores
 - ▶ `\p{S}` o `\p{Symbol}`, símbolos matemáticos, de monedas, etc.
 - ▶ `\p{N}` o `\p{Number}`, números
 - ▶ `\p{P}` o `\p{Punctuation}`, símbolos de puntuación.
- P. ej. la clase POSIX `[:punct:]`, que contiene los símbolos de puntuación se corresponde con las clases Unicode `\p{P}` y `\p{S}`, con lo que podemos usar en lugar de `[:punct:]` la clase `[\p{P}\p{S}]`, que es preferible.

Paréntesis en regex

- Se pueden crear clases de caracteres usando el operador "|", que es el "o" lógico; "a|f|H" significa o el carácter "a", o el "f", o el "H", así que es lo mismo que "[afH]"

```
> grepl2("semáforo",c("a|f|H","x|t|g"))
[1] TRUE FALSE
```

- Si tenemos "a|fo|H", significa que el *match* es con "a", o con "fo" o con "H".
- Se pueden usar paréntesis en las expresiones regulares igual que en matemáticas, para agrupar términos: "(casa|chalet) grande" hace *match* con "casa grande" y con "chalet grande":

```
> grepl2("tiene un chalet grande","(casa|chalet) grande")
[1] TRUE
```

- Esto es un *capturing parenthesis*, ya que captura el término con el que hace *match* ("chalet" en este caso), que estará disponible en una variable para poder usarlo posteriormente en la *regex*.

Paréntesis en regex

- El término capturado por el paréntesis quedará almacenado en la variable `\1`, y si hay más *capturing parenthesis* en la *regex*, los términos capturados por ellos quedarán almacenados sucesivamente en las variables `\2` (para el segundo paréntesis), `\3` (para el tercer paréntesis), etc. También se les pueden poner nombres a estas variables.
- En el ejemplo siguiente, el string "3" con el que hace *match* la *regex* `"\\d"` quedará almacenado en la variable `\1`, que podrá ser usada más adelante en la *regex*:

```
> grepl2("le han puesto 3 multas", "(\\d) multa")
[1] TRUE
```

- Podemos hacer referencia (esto es una **backreference**) a la variable `\1` en la *regex* (en R habrá que usar `\\1`):

```
> grepl2("le han puesto 3 multas; no se quitan puntos
        si no tienes más de 2", "(\\d) multas.+\\1")
[1] FALSE
```

Paréntesis en regex

- Otro ejemplo:

```
> grepl2("tiene un chalet grande; el chalet es bonito",  
         "(casa|chalet) grande.+\\1.+bonit(?:o|a)")  
[1] TRUE
```

- Un *non-capturing parenthesis* simplemente agrupa términos, pero no captura el *string* con el que se hace el *match*. Se indica con `(?: )`, p. ej. `"(?:casa|chalet) grande"`.

Cuantificadores

- En los ejemplos anteriores, dentro de la *regex* aparecen los caracteres `".+"`.
- `"."` es un operador que indica cualquier carácter *una* vez; para indicar una o más veces se le añade a continuación el operador `"+"` (de forma similar `"(ac)+b"` hace *match* con `"acacacb"`).
- Si queremos especificar el número de veces que se repite un *string* usamos el operador `{n,m}` que indica una repetición del carácter o grupo de caracteres precedente entre *n* y *m* veces:

```
> grepl2("tiene un chalet grande; el chalet es bonito",
         "(casa|chalet)grande.+\\1.{1,20}bonit(?:o|a)")
[1] TRUE
```

- Existen varios **cuantificadores** de este tipo, que permiten indicar el número de veces que un carácter o grupo de caracteres aparece repetido en una *regex*. Van siempre detrás del carácter que se repite.

Cuantificadores

- Los cuantificadores fundamentales son:
 - ▶ `*`, repetición cero o más veces
 - ▶ `+`, repetición una o más veces
 - ▶ `?`, repetición cero o una vez (preferible una)
 - ▶ `{n}`, repetición de n veces
 - ▶ `{n,}`, repetición de n veces o más
 - ▶ `{n,m}`, repetición entre n y m veces
- Todos estos cuantificadores se denominan *greedy* (voraces), en el sentido de que intentan buscar el ajuste con el mayor número de caracteres posible posible.
- P. ej, la *regex* `".*"` hace *match* con una línea completa, y si está activado el *flag dotall*, con un *string* completo, aunque tenga varias líneas:

```
> stri_extract_all_regex("coche\nbonito",".*")
[[1]]
[1] "coche"      ""           "bonito"     ""
> stri_extract_all_regex("coche\nbonito","(?s).*")
[[1]]
[1] "coche\nbonito" ""
```

Cuantificadores

Cuantificadores lazy

- Los **cuantificadores greedy** tienen una versión **lazy**, que selecciona el *substring* más corto posible con el que se hace *match*.
- Los cuantificadores *lazy* se indican igual que los *greedy*, pero añadiéndoles un símbolo «?» detrás:
 - ▶ p. ej $\{n,m\}?$ indica entre n y m repeticiones pero el menor número posible de ellas (no menor que n , obviamente)
 - ▶ $\{n\}?$ será igual que $\{n\}$ (n repeticiones exactamente)

```
> gregexpr2("aaa3f", "a+")
[[1]]
[1] 1
attr(,"match.length")
[1] 3
> gregexpr2("aaa3f", "a+?")
[[1]]
[1] 1 2 3
attr(,"match.length")
[1] 1 1 1
```

Cuantificadores

Cuantificadores posesivos

- Un cuantificador *greedy* usa el mayor número posible de repeticiones para intentar el *match* global teniendo en cuenta el resto de la *regex*. Si no lo consigue, usa una repetición menos y vuelve a intentar el *match*. Si no lo consigue vuelve a reducir el número de repeticiones, y vuelve a intentar el *match* global, y así sucesivamente.
- Todo este proceso de marcha atrás (*backtracking*) puede ser muy costoso computacionalmente.
- Los **cuantificadores posesivos** (*possessive*) son una variante de los *greedy* que no hacen *backtracking*, intentan un *match* global con el mayor número posible de repeticiones y, si no lo consiguen, devuelven un *no-match*.
- Se indican añadiendo un "+" al final del cuantificador:

```
> x <- "bca3f"
> substr1(x, regexpr2(x, "[a-z]+(a\\d)"))
[1] "bca3"
> substr1(x, regexpr2(x, "[a-z]++(a\\d)"))
[1] NA
```


Funciones con *regex*

- La función ***regexpr2()*** es el equivalente en *stringx* a *regexpr()* de R base; se usa para detectar la posición de la de un patrón dentro de un *string*.
- Se le pasa un vector de caracteres y un *pattern*, devuelve un vector con la posición de la primera ocurrencia del patrón en cada componente del vector de caracteres (o -1 si no hay *match*) y con un atributo que es la longitud del *match* en caracteres (o -1 si no hay *match*):

```
> regexpr2(c("pepe", "vaca"), "pe")
[1] 1 -1
attr(,"match.length")
[1] 2 -1
```

- Si la posición del *match* es *n* y su longitud es *m* significa que el *substring* con el que hace *match* la *regex* se obtiene situándose **justo antes del carácter *n*** y contando *m* caracteres a partir de ahí.

Funciones con *regex*

- Son posibles ***matches* de longitud cero**:

```
> regexpr2("pepe", "\\d?")
```

```
[1] 1
```

```
attr(,"match.length")
```

```
[1] 0
```

- De hecho son posibles *matches* de longitud cero al final de un *string*, cuando ya no queda ningún carácter después:

```
> grepl2("", "\\d*")
```

```
[1] TRUE
```

- La mecánica para obtener los *matches* es la siguiente:
 - ▶ nos situamos al comienzo del *string*, justo antes del primer carácter
 - ▶ buscamos un *substring* que empiece ahí y con el que la *regex* haga *match* (el *substring* más largo posible si usamos cuantificadores *greedy*)
 - ▶ si se encuentra saltamos a la posición justo después del *match*
 - ▶ si no se encuentra avanzamos una posición
 - ▶ vamos repitiendo hasta llegar al fin del *string*
- De esta manera **los sucesivos *matches* no se solapan**.

Funciones con *regex*

- Si queremos obtener las posiciones y longitudes de **todos** los *matches* dentro de cada *string* usamos la función *gregexpr2()*, que devuelve una lista con un componente para cada elemento del vector de caracteres:

```
> gregexpr2(c("pepe", "vaca"), "pe")
```

```
[[1]]
```

```
[1] 1 3
```

```
attr(,"match.length")
```

```
[1] 2 2
```

```
[[2]]
```

```
[1] -1
```

```
attr(,"match.length")
```

```
[1] -1
```

- Es habitual en los motores de *regex* que por defecto devuelvan el primer *match* y que se use un modificador «g» para que los devuelvan todos.

Funciones con *regex*

- Cuando se produce un *match* de longitud cero, si saltamos al final del *match* nos quedamos en el mismo sitio, con lo cual no avanzamos a lo largo del *string* y entramos en un bucle infinito.
- Entonces lo habitual es que tras un *match* de longitud cero se avance una posición (aunque hay otras posibilidades):

```
> gregexpr2("x1x", "\\d*")  
[[1]]  
[1] 1 2 3 4  
attr(,"match.length")  
[1] 0 1 0 0
```

Funciones con *regex*

- Para extraer los ***matches***, es decir los *substrings* con los que encajan las *regex*, podemos usar *substr()* y *gsubstr()*, pero estas usan la posición inicial y final (*start* y *stop*) de cada *substring* y es más cómodo usar ***substrl()*** y ***gsusbsl()***, que usan la posición inicial y la longitud de cada *substring*, tal como las devuelven *regexpr2()* y *gregexpr2()*:

```
> gsubsl(x,gregexpr2(x,c("hi","(?i)(pe)+")))
[[1]]
[1] "hi"
[[2]]
[1] "Pepe"
```

- El cuantificador "+" indica una repetición o más del grupo de caracteres precedente, y es *greedy*, ¿qué pasa si usamos su versión *lazy*?:

```
> gsubsl(x,gregexpr2(x,c("hi","(?i)(pe)+?")))
[[1]]
[1] "hi"
[[2]]
[1] "Pe" "pe"
```

Funciones con *regex*

- **Para reemplazar caracteres** que sigan un determinado patrón dentro de un vector de *strings* podemos usar las funciones ***sub2()*** y ***gsub2()***, cuya sintaxis es:

```
sub2(x, pattern, replacement,...)  
gsub2(x, pattern, replacement,...)
```

x es el vector de strings donde se hacen las substituciones

pattern es un regex que indica los substrings que serán reemplazados

replacement es un vector con los strings que se usarán como reemplazo

- Como siempre *sub2()* reemplaza la primera ocurrencia en cada *string* y *gsub2()* las reemplaza todas.
- Otra posibilidad, que ya hemos mencionado, es usar *substr()* con una asignación.

Funciones con *regex*

```
> gsub2("Pepe lo tramó, lo aprobó Pepón, lo ejecutó Pepín",
        "(?i)(\\b)pep\\p{Alphabetic}+\\b", "Manolo")
[1] "Manolo lo tramó, lo aprobó Manolo, lo ejecutó Manolo"
```

- Vamos a crear un *regex* para eliminar todos los espacios en blanco duplicados y al principio o al final de un *string*.
- P.ej, el siguiente *string*:

```
> t1 <- c(" La sopa estaba fría cuando la abuela llegó a casa ")
```

- Lo lógico es ir por pasos, primero eliminar los espacios en blanco al principio y al final, ¿cómo lo hacemos?.
- Y con el resultado luego eliminar los espacios en blanco múltiples.
- También existe una función *trimws()* que elimina los espacios en blanco al principio y al final.

Operadores lookahead

- Otros operadores importantes en las *regex* son los ***lookaround***, que pueden ser ***lookahead*** (búsqueda hacia adelante) o ***lookbehind*** (búsqueda hacia atrás).
- Cuando hay uno de estos operadores dentro de una *regex* lo que se hace es, partiendo de la posición del *string* en la que nos encontremos, buscar hacia adelante (en un *lookahead*) o hacia atrás (en un *lookbehind*) para intentar encontrar en el *string* el contenido del *lookaround*. Si se encuentra, continuamos con el resto de la *regex* para intentar encontrar un *match* global.
- Realmente esto es como con cualquier *regex*, lo peculiar cuando hay un *lookaround* es que:
 - ▶ si se encuentra un *match* para el *lookaround*, no se avanza la posición en el *string* a la de después del *match*, sino que nos mantenemos en la misma posición, la que había al comprobar el *match* del *lookaround*
 - ▶ el *match* del *lookaround* se descarta del *substring* final que se obtiene con la *regex*

Operadores lookahead

- Si queremos buscar "ABC" con un *lookahead* lo indicamos con "(?=ABC)"


```
> substr1("abcdABCDE", regexpr2("abcdABCDE", "cd"))
[1] "cd"
> substr1("abcdABCDE", regexpr2("abcdABCDE", "cd(?=ABC)"))
[1] "cd"
> substr1("abcdABCDE", regexpr2("abcdABCDE", "cd(?=ABC)[A-Z]{2}"))
[1] "cdAB"
```
- "(?!ABC)" es un *lookahead* negativo: buscamos algo que no sea "ABC"
- "(?<=ABC)" es un *lookbehind*: buscamos "ABC" justo antes de la posición actual
- "(?<!ABC)" es un *lookbehind* negativo: buscamos algo que no sea "ABC" justo antes de la posición actual
- Los paréntesis de los *lookahead* son *non-capturing*.

Operadores lookahead

- Supongamos que tenemos el *string* "camaradería" y le aplicamos la *regex* "(?<=a)r", que es un *lookbehind* que busca una «r» precedida por una «a».
- Vamos recorriendo el *string* camaradería sin encontrar *matches* hasta que llegamos a la posición justo antes de la primera «r»: *cama* [↑]*radería*; en esa posición la *regex* busca una «a» detrás y una «r» a continuación, que existen, con lo cual tenemos el *match* y la *regex* nos devolvería un *match* con el *substring* "r" (la «a» del *match* del *lookbehind* se descarta, ya que los *matches* de los *lookaround* son siempre de longitud cero):

```
> substr1("camaradería",regexpr2("camaradería","(?<=a)r"))
[1] "r"
```

Operadores lookahead

- Si usamos la *regex* "(?<!a)r", que es un *lookbehind* negativo, lo que se busca es una «r» que no esté precedida por una «a», con lo cual el *match* será con la ultima «r» y la «e» que la precede, es decir que se producirá cuando lleguemos a la siguiente posición: camarade [↑]ría.
- La *regex* nos devolverá una «r» igual que antes, pero extraída de una posición distinta:

```
> substr1("camaradería",regexpr2("camaradería","(?<!a)r"))
[1] "r"
```

- Supongamos que tenemos el *string* "QuéBonitoEsTodoEsto", donde las palabras empiezan por mayúscula y están pegadas sin espacios en blanco. Si queremos separar las palabras tenemos que identificar los límites entre ellas, que serán las posiciones en las que existe una letra minúscula seguida por una letra mayúscula.

Operadores lookahead

- Podemos usar las clases *unicode* `\p{Lu}` y `\p{Ll}` para referirnos a las letras mayúsculas y a las minúsculas, respectivamente.
- Podemos también usar un *lookbehind* y otro *lookahead* para identificar las posiciones que tienen hacia atrás una letra minúscula y hacia adelante una mayúscula.
- Construimos una regex que empieza con una mayúscula, seguida de minúsculas hasta que llega a una posición en la que hacia atrás hay minúscula y hacia adelante mayúscula:

```
> x <- "QuéBonitoEsTodoEsto"
> gsubstr1(x,gregexpr2(x,"\\p{Lu}\\p{Ll}+(?<=\\p{Ll})(?=\\p{Lu})"))
```

- ¿Qué problema tiene ésto?
- ¿Cómo lo corregimos?

Ejemplo

- Vamos a ver un ejemplo de cómo validar *passwords* usando *regex* con operadores *lookaround*.
- Supongamos que las *passwords* tienen que cumplir una serie de requisitos, como tener entre 8 y 12 caracteres (de la clase `\w`), de ellos tiene que haber al menos 2 mayúsculas, 3 minúsculas y 2 dígitos.
- P. ej. tenemos una *password* que es el *string* "B1252flhuH" y queremos comprobar si es una *password* válida usando una *regex* que haga *match* con la *password* si es válida y sólo en ese caso.
- Podemos chequear la condición de que tenga entre 8 y 12 caracteres `\w` con la siguiente *regex*, "`\\A(?:\\w{8,12}\\z)`", que hará *match* si la *password* tiene entre 8 y 12 caracteres del tipo requerido, pero no moverá la posición en la que buscamos en el *string*, que seguirá justo al principio.

```
> x <- "B1252flhuH"
> grep12(x, "\\A(?:\\w{8,12}\\z)")
[1] TRUE
```

Ejemplo

- Ampliamos la *regex* para comprobar que contiene al menos 2 mayúsculas (categoría Lu en *unicode*). Para ello añadimos un *lookbehind* a la *regex*: `(?=(\\P{Lu}*\\p{Lu})){2,}`:

```
> grep12(x, "\\A(?=\\w{8,12}\\z)(?=(\\P{Lu}*\\p{Lu})){2,}")
[1] TRUE
```

- Esto funciona porque tras el primer *lookbehind* volvemos a buscar desde el principio, con lo que podemos buscar dos mayúsculas, al menos, a lo largo de toda la *password*.
- Además las dos mayúsculas no tiene por qué estar juntas, así que tenemos que buscar dos grupos de una serie de no mayúsculas seguidas por una mayúscula.
- La *regex* se puede mejorar, podemos hacer *non-capturing* el paréntesis `(\\P{Lu}*\\p{Lu})`, podemos usar `{2}` en lugar de `{2,}`, etc.

Ejemplo

- Luego añadimos otro *lookbehind* similar para comprobar que hay al menos tres letras minúsculas: $(?=(?:\\P{Ll}^*\\p{Ll})\\{3})$:

```
> grepl2(x, "\\A(?=\\w{8,12}\\z)(?=\\P{Lu}^*\\p{Lu}){2,}(?=\\P{Ll}^*\\p{Ll}){3})")
```

```
[1] TRUE
```

- Por último, comprobamos que la *password* tiene al menos 2 dígitos añadiendo el lookbehind $(?=(?:\\P{N}^*\\p{N})\\{2})$:

```
> condic <- "\\A(?=\\w{8,12}\\z)(?=\\P{Lu}^*\\p{Lu}){2,}(?=\\P{Ll}^*\\p{Ll}){3})(?=\\P{N}^*\\p{N}){2})"
```

```
> grepl2(x, condic)
```

```
[1] TRUE
```

Bibliografía



Friedl, Jeffrey E. F. (2006). *Mastering Regular Expressions*. 3rd ed. Sebastopol, California: O'Reilly.



Gagolewski, Marek (2022). “stringi: Fast and Portable Character String Processing in R”. En: *Journal of Statistical Software* 103.2, págs. 1-59. DOI: [10.18637/jss.v103.i02](https://doi.org/10.18637/jss.v103.i02). URL: <https://www.jstatsoft.org/index.php/jss/article/view/v103i02>.



Sánchez, Gastón (2024). *R for strings. Handling Strings With R*. URL: <https://www.gastonsanchez.com/R-for-strings/> (visitado 26-12-2024).